

Noesis Tension: A Telemetry-Driven Taxonomy of Prompt-Induced Representational Pressures in Large Language Models

James Benjamin Jones
ORCID: 0009-0002-6129-2847
Independent Researcher

April 2026

Abstract

We introduce Noesis Tension, a lightweight, telemetry-only framework for diagnosing how prompts induce specific representational pressures inside large language models. Using only per-layer activation deltas, curvature, entropy, and a compact set of twelve empirically-derived “tension categories,” the system computes quantitative indices of tension and drift and classifies the model into one of several cognitive regimes (e.g., `exploratory_liminal`, `safety_procedural`, `factual_stable`, `confident_hallucination_lite`).

Across three architecturally distinct models (Llama-3.1-8B, Mistral-7B, Qwen1.5-MoE-2.7B) and diverse prompts—including false-historical summaries—we show that the primary representational attractor is stable, while secondary categories capture model-specific flavor. The framework requires no prompt or response text and runs in microseconds, providing an immediately actionable diagnostic for mechanistic interpretability and safety evaluation. We release the classifier code and example traces to support further research.

1 Introduction

Modern safety and alignment work on large language models has largely focused on observable outputs (refusals, harm scores) or low-level circuit analysis. While valuable, these approaches often miss the *intermediate representational dynamics* that determine why a model suddenly slips into creative derailment, confident hallucination, or ungrounded conspiracy reasoning.

We present Noesis Tension, a practical telemetry-driven taxonomy that bridges this gap. By extracting only activation statistics (per-layer mean/median/p95 deltas, curvature, entropy, and final-layer spike behavior), the system quantifies two core axes—tension and drift—and maps them onto a compact set of twelve prompt-induced “tension categories.” These categories (Moral Paradox Tension, Ontological Impossibility, Ungrounded Causality, Emergent Symbolism Amplification, etc.) consistently surface the same core attractor across model families on identical prompts, while secondary categories reveal interpretable architectural differences.

Building on earlier conceptual work that introduced the need for epistemic regime detection [1, 2], this paper presents the first concrete telemetry-only implementation of the Noesis Tension framework.

The resulting cognitive regime labels (`exploratory_liminal`, `safety_procedural`, `confident_hallucination_lite`, etc.) provide a high-level yet mechanistically grounded vocabulary for describing model behavior. In this paper we detail the framework, validate it on cross-model experiments, and demonstrate its utility for safety diagnostics and comparative mechanistic interpretability.

2 Related Work

Noesis Tension occupies the middle ground between low-level mechanistic interpretability and high-level safety evaluation. Early work on induction heads (Olsson et al., 2022) and sparse autoencoders (Bricken et al., 2023) demonstrated that internal activations contain interpretable structure, but typically requires heavy per-behavior reverse-engineering. Hallucination surveys (Ji et al., 2023) and latent knowledge probes (Azaria & Mitchell, 2023; Burns et al., 2024) focus on detecting ungrounded or untruthful outputs, usually after generation. Behavioral regime discovery (Perez et al., 2022; Zou et al., 2023; Turner et al., 2024) has shown that LLMs exhibit distinct modes (e.g., sycophantic vs. honest), but these are often identified via output clustering or steering vectors rather than lightweight telemetry. Red-teaming frameworks (Ganguli et al., 2022; Mazeika et al., 2024) classify prompts by surface strategy or output harm.

Noesis Tension differs by providing a telemetry-only, prompt-sensitive taxonomy that quantifies representational pressure directly from per-layer statistics, yielding stable cross-model categories and cognitive regime labels without inspecting generated text.

3 Methods

3.1 Telemetry Backbone and Noesis Category Scorer

Noesis Tension operates exclusively on per-layer activation telemetry. For each generation run we extract mean, median, and p95 absolute deltas, tail ratios, curvature, logit entropy, and final-layer spike magnitude. These raw signals feed into the `telemetry_noesis_profile` function (v0.1), which computes a 12-dimensional category vector using simple linear heuristics on normalized features (tension, drift, curvature, entropy, max-delta, and optional MoE routing anomaly). The function is telemetry-only and produces stable, reproducible scores.

3.2 Cognitive Regime Classifier

The category vector, tension/drift indices, and entropy statistics are passed to `infer_cognitive_regime`, which applies a priority-ordered set of heuristic rules to assign a deterministic regime label (`exploratory_liminal`, `safety_procedural`, `factual_stable`, `confident_hallucination_lite`, etc.) plus a natural-language explanation. A secondary function `compute_stability_v0` produces probabilistic support scores for every regime and reports the margin between the top-1 and top-2 regimes, along with a bucket (`stable` / `boundary` / `indeterminate`) and an honesty flag `label_matches_top`. Both functions are strictly telemetry-only, operate on pre-computed numeric features, and run in microseconds.

Full classifier code is provided in Appendix A.

4 Results

We evaluated the framework on three architecturally distinct models using the identical prompt:

“Summarize the political reforms enacted by the Roman Emperor Aurelius Magnus in 312 CE.”

This is a pure false-presupposition prompt (no such emperor existed). All three models produced confident but fabricated historical narratives.

Ontological Impossibility (category 2) and Aesthetic-Logic Crosswiring (category 10) appeared in the top three for all models, showing a stable core attractor despite regime variation. The framework reliably flags the representational pressure induced by the false historical premise.

Table 1: Cross-model regime and Noesis profile for the Aurelius Magnus prompt.

Model	Regime	Tension	Drift	Top-3 Categories	Final Entropy
Mistral-7B	confident_hallucination_lite	0.332	0.688	2, 7, 10	0.338
Qwen1.5-MoE-2.7B	mixed_unclear	0.464	0.492	4, 10, 2	2.619
Llama-3.1-8B	exploratory_liminal	0.441	0.542	9, 10, 11	4.574

5 Discussion and Limitations

Implications. Noesis Tension reveals that many harmful or deranged behaviors arise not from direct alignment overrides but from a shift into the `exploratory_liminal` or `confident_hallucination_lite` regime. Once the model enters these regimes, guardrails that function well under procedural reasoning simply never activate. The framework therefore offers a practical early-warning signal: monitor tension/drift indices and the Noesis profile vector in real time.

The taxonomy also provides a new comparative lens for mechanistic interpretability. The consistent primacy of certain attractors (e.g., Ontological Impossibility or Paradox Pressure) across models, while secondary categories diverge in model-specific ways, suggests that Noesis categories capture genuine differences in representational geometry.

Limitations. The current heuristics (v0.1) use hand-tuned linear combinations of normalized telemetry features. While they generalize well across the models tested, broader validation on larger model families and more diverse prompt packs is needed. Some regimes have not yet been strongly triggered in our corpus and remain provisional.

The `stability_v0` margin analysis is intentionally lightweight and does not yet incorporate uncertainty from the underlying deltas. The framework currently operates on a per-run basis; extending it to token-level or streaming regime tracking is a natural next step.

Future work. We plan to open-source the full Noesis classifier pipeline and all telemetry traces. We will expand the prompt packs and investigate integration of Noesis signals with sparse autoencoders and circuit-level analysis to further ground the tension categories in mechanistic explanations.

References

- [1] James Benjamin Jones. NOESIS Phase I: Toward Epistemic Regime Detection. Zenodo, December 2025. <https://zenodo.org/records/17917279>
- [2] James Benjamin Jones. NOESIS Phase II: Observability Architecture for Cognitive Telemetry. Zenodo, January 2026. <https://zenodo.org/records/18141914>

A Classifier Code

The complete, production-ready implementation (strictly telemetry-only, no prompt/response text logic) is given below.

```

1 def telemetry_noesis_profile(
2     metrics: TensionMetrics,
3     hti_v2: Optional[Dict[str, float]] = None,
4     moe_summary: Optional[Dict[str, Any]] = None,
5 ) -> Dict[str, Any]:
6     """
7     Telemetry-only Noesis category scorer (v0.1).
8     IMPORTANT: uses numbers only (no prompt/response string logic).
9     Output is stable for analytics; labels are derived at render time.
10    """

```

```

11 # Core numeric features
12 tension = (
13     float(hti_v2.get("tension_index"))
14     if hti_v2 and hti_v2.get("tension_index") is not None
15     else None
16 )
17 drift = (
18     float(hti_v2.get("drift_index"))
19     if hti_v2 and hti_v2.get("drift_index") is not None
20     else None
21 )
22
23 mean_delta = float(metrics.mean_layer_delta or 0.0)
24 max_delta = float(metrics.max_layer_delta or 0.0)
25 mean_entropy = float(metrics.mean_logit_entropy or 0.0)
26 final_entropy = float(metrics.final_token_entropy or 0.0)
27 mean_curv = float(metrics.mean_curvature or 0.0)
28 max_curv = float(metrics.max_curvature or 0.0)
29
30 # MoE numeric features (optional)
31 moe_mean_ent = None
32 moe_max_ent = None
33 moe_anom = None
34 if moe_summary:
35     moe_mean_ent = moe_summary.get("mean_routing_entropy", None)
36     moe_max_ent = moe_summary.get("max_routing_entropy", None)
37     an = moe_summary.get("anomaly", None) or {}
38     moe_anom = an.get("score_v0_1", None)
39
40 # Helper scaling
41 def clamp01(x: float) -> float:
42     return float(max(0.0, min(1.0, x)))
43
44 def nz(x, default=0.0):
45     return default if x is None else float(x)
46
47 t = nz(tension, 0.0)
48 d = nz(drift, 0.0)
49 mc = clamp01(mean_curv / 1.2)
50 xc = clamp01(max_curv / 1.4)
51 me = clamp01(mean_entropy / 8.0)
52 fe = clamp01(final_entropy / 8.0)
53 xd = clamp01(max_delta / 150.0)
54 mma = clamp01(nz(moe_anom, 0.0))
55
56 # --- Telemetry-only heuristics (v0.1) ---
57 scores = [0.0] * 12
58
59 scores[0] = clamp01(1.4 * t * (1.0 - d) * (1.0 - fe)) #
60 Moral Paradox Tension #
61 scores[1] = clamp01(1.2 * d * (1.0 - t) * (1.0 - fe)) #
62 False Presupposition Buckling #
63 scores[2] = clamp01(0.6 * d + 0.25 * xc + 0.15 * xd) #
64 Ontological Impossibility #
65 scores[3] = clamp01(0.5 * d + 0.3 * t + 0.2 * (1.0 - mc)) #
66 Anthropomorphic Metaphor Pressure #
67 scores[4] = clamp01(0.35 * mc + 0.35 * mma + 0.3 * d) #
68 Category Collision #
69 scores[5] = clamp01(0.7 * t + 0.2 * (1.0 - fe) + 0.1 * xd) #
70 Alignment Override Request #
71 scores[6] = clamp01(0.6 * d + 0.25 * fe + 0.15 * (1.0 - t)) #
72 Emergent Symbolism Amplification

```

```

66     scores[7] = clamp01(0.5 * d + 0.25 * mc + 0.25 * me) #
    Ambiguous Containment
67     scores[8] = clamp01(0.35 * t + 0.35 * d + 0.3 * me) #
    Ontology Blur / Identity Bleed
68     scores[9] = clamp01(0.4 * mc + 0.35 * fe + 0.25 * d) #
    Paradox Pressure
69     scores[10] = clamp01(0.45 * mc + 0.35 * me + 0.2 * (1.0 - t)) #
    Aesthetic-Logic Crosswiring
70     scores[11] = clamp01(0.55 * d + 0.3 * fe + 0.15 * mc) #
    Ungrounded Causality
71
72     total = float(sum(scores))
73     vector = [s / total for s in scores] if total > 1e-9 else [0.0] * 12
74
75     indexed = sorted(list(enumerate(vector)), key=lambda x: x[1], reverse=True)
76     top_indices = [idx for idx, _ in indexed[:3]]
77
78     return {
79         "vector": vector,
80         "top_indices": top_indices,
81         "raw_scores": scores,
82         "features_v0_1": {
83             "tension": tension,
84             "drift": drift,
85             "mean_delta": mean_delta,
86             "max_delta": max_delta,
87             "mean_entropy": mean_entropy,
88             "final_entropy": final_entropy,
89             "mean_curvature": mean_curv,
90             "max_curvature": max_curv,
91             "moe_anomaly": moe_anom,
92             "moe_mean_entropy": moe_mean_ent,
93             "moe_max_entropy": moe_max_ent,
94         },
95     }

```

Listing 1: telemetry_noesis-profile (v0.1)

```

1 def infer_cognitive_regime(
2     metrics: TensionMetrics,
3     noesis_profile: Dict[str, Any],
4     tension_index: Optional[float],
5     drift_index: Optional[float],
6     is_rap: bool,
7     is_safety_liminal: bool,
8     regime_baseline: Optional[Dict[str, float]] = None,
9 ) -> Dict[str, Any]:
10     """
11     Classify a high-level 'cognitive regime' for this trace using:
12     - HTI v0.2 (tension vs drift)
13     - Noesis category profile
14     - Safety/RAP flags
15     Returns:
16     {"label": str, "explanation": str, "features": {...}}
17     """
18     def safe(x, default=0.5):
19         return float(x) if x is not None else default
20
21     def clamp01(x: float) -> float:
22         if x < 0.0: return 0.0
23         if x > 1.0: return 1.0
24         return x
25
26     tension = safe(tension_index, 0.5)

```

```

27     drift = safe(drift_index, 0.5)
28
29     vec = noesis_profile.get("vector") or [0.0] * 12
30
31     # Noesis category shorthand
32     moral_paradox = vec[0]
33     false_presupposition = vec[1]
34     category_collision = vec[4]
35     symbolism = vec[6]
36     ambiguous_containment = vec[7]
37     paradox_pressure = vec[9]
38
39     # Simple banding
40     high_tension = tension >= 0.60
41     low_tension = tension <= 0.40
42     high_drift = drift >= 0.60
43     mid_drift = 0.45 <= drift < 0.60
44     low_drift = drift < 0.45
45
46     high_moral = moral_paradox >= 0.25
47     high_fp = false_presupposition >= 0.25
48     high_symbolism = symbolism >= 0.25
49     high_paradox = paradox_pressure >= 0.25
50     high_ambiguity = ambiguous_containment >= 0.25
51
52     low_moral = moral_paradox < 0.18
53     low_fp = false_presupposition < 0.18
54     low_symbolism = symbolism < 0.18
55
56     # Entropy shape helpers
57     mean_entropy = float(getattr(metrics, "mean_logit_entropy", 0.0) or 0.0)
58     final_entropy = float(getattr(metrics, "final_token_entropy", 0.0) or 0.0)
59     entropy_collapse = (mean_entropy > 1e-9) and (final_entropy < 0.85 *
60     mean_entropy)
61     entropy_sustained = (mean_entropy > 1e-9) and (final_entropy >= 0.95 *
62     mean_entropy)
63
64     symbolism_score = max(symbolism, ambiguous_containment, paradox_pressure,
65     category_collision)
66     low_drift_score = clamp01((0.65 - drift) / 0.65)
67
68     # Baseline-relative factual banding
69     in_factual_control_band = False
70     z_t = None
71     z_d = None
72     if regime_baseline:
73         try:
74             t_mu = float(regime_baseline.get("factual_tension_mu", 0.5))
75             t_sd = float(regime_baseline.get("factual_tension_sigma", 1.0)) or
76             1.0
77             d_mu = float(regime_baseline.get("factual_drift_mu", 0.5))
78             d_sd = float(regime_baseline.get("factual_drift_sigma", 1.0)) or
79             1.0
80             if t_sd == 0.0: t_sd = 1.0
81             if d_sd == 0.0: d_sd = 1.0
82             z_t = (tension - t_mu) / t_sd
83             z_d = (drift - d_mu) / d_sd
84             in_factual_control_band = (abs(z_t) <= 2.0) and (abs(z_d) <= 2.0)
85             factual_tension_ok = abs(z_t) <= 2.0
86         except Exception:
87             factual_tension_ok = 0.35 <= tension <= 0.65
88     else:
89         factual_tension_ok = 0.35 <= tension <= 0.65

```

```

85
86 # ---- Priority: hard archetype overrides ----
87 if is_rap:
88     return {"label": "reflexive_alignment_panic", "explanation": "Early
conflict + refusal + large deltas indicate a safety panic pattern.", "
features": {"tension": tension, "drift": drift, "noesis_top": noesis_profile
.get("top_indices")}}
89
90 if is_safety_liminal:
91     return {"label": "safety_liminality", "explanation": "High tension, low
drift, late-layer spike, and explicit refusal signal an alignment override.",
"features": {"tension": tension, "drift": drift, "noesis_top":
noesis_profile.get("top_indices")}}
92
93 # ---- CTRL / Null Conditions Guard ----
94 if metrics.label == "class_a" and regime_baseline and
in_factual_control_band:
95     return {"label": "factual_stable", "explanation": "CTRL band: within
baseline-normal z-band for class_a.", "features": {"tension": tension, "
drift": drift, "z_tension": z_t, "z_drift": z_d, "entropy_collapse":
entropy_collapse}}
96
97 # ---- Regime 1: Ethical Paradox ----
98 if high_moral and high_tension and (mid_drift or low_drift):
99     return {"label": "ethical_paradox", "explanation": "Strong moral-
paradox signal with high tension and controlled drift.", "features": {"
tension": tension, "drift": drift, "moral_paradox": moral_paradox}}
100
101 # ---- Regime 2: False-Premise Buckling ----
102 if high_fp and (mid_drift or high_drift) and not entropy_collapse:
103     return {"label": "false_premise_buckling", "explanation": "Model treats
a broken premise as real with high drift and low internal tension.", "
features": {"tension": tension, "drift": drift, "false_presupposition":
false_presupposition}}
104
105 # ---- Regime 3: Mythic Drift ----
106 if high_symbolism and (high_drift or mid_drift) and not high_moral:
107     return {"label": "mythic_drift", "explanation": "Symbolic or mythic
content amplified under high drift without strong moral tension.", "features
": {"tension": tension, "drift": drift, "symbolism": symbolism}}
108
109 # ---- Regime X: Confident Hallucination ----
110 if entropy_collapse and drift > 0.3 and false_presupposition >= 0.12 and
not is_rap and not is_safety_liminal:
111     return {"label": "confident_hallucination", "explanation": "Entropy
collapse with narrative commitment despite weak explicit false-
presupposition signal.", "features": {"tension": tension, "drift": drift, "
false_presupposition": false_presupposition, "entropy_collapse":
entropy_collapse}}
112
113 # ---- Regime X.5: Confident Hallucination Lite ----
114 if entropy_collapse and drift > 0.45 and false_presupposition >= 0.06 and
low_moral and not is_rap and not is_safety_liminal:
115     return {"label": "confident_hallucination_lite", "explanation": "
Entropy collapse with moderate drift and a weak-to-moderate false-
presupposition signal; model is committing confidently to a likely
fabricated frame.", "features": {"tension": tension, "drift": drift, "
false_presupposition": false_presupposition, "entropy_collapse":
entropy_collapse}}
116
117 # ---- Regime 5: Safety / Procedural Reasoning ----
118 if low_drift and not high_moral and not high_fp:
119     return {"label": "safety_procedural", "explanation": "Low drift and

```

```

modest tension with little paradox or false-premise signal; likely
procedural reasoning.", "features": {"tension": tension, "drift": drift}}
120
121 # ---- Regime 6: Factual / Stable ----
122 if low_drift and factual_tension_ok and low_fp and low_symbolism and
low_moral and not entropy_collapse:
123     return {"label": "factual_stable", "explanation": "Balanced tension and
drift with weak hallucination and symbolic signals; behavior looks factual
and grounded.", "features": {"tension": tension, "drift": drift}}
124
125 # ---- Regime Y: Exploratory Liminal ----
126 if drift > 0.5 and false_presupposition < 0.3 and (entropy_sustained or (
symbolism_score > low_drift_score)):
127     return {"label": "exploratory_liminal", "explanation": "Elevated drift
with sustained entropy OR symbolic-dominance; behavior looks exploratory
rather than fact-committing.", "features": {"tension": tension, "drift":
drift, "symbolism": symbolism, "ambiguous_containment":
ambiguous_containment, "paradox_pressure": paradox_pressure, "mean_entropy":
mean_entropy, "final_entropy": final_entropy, "entropy_sustained":
entropy_sustained}}
128
129 # ---- Regime Z: Procedural / Ethical Oddity ----
130 if (high_drift or mid_drift) and low_fp and high_moral and not is_rap and
not is_safety_liminal:
131     return {"label": "procedural_oddity", "explanation": "Moral/
justification pressure under drift without a strong false-premise signature.
", "features": {"tension": tension, "drift": drift, "moral_paradox":
moral_paradox}}
132
133 # ---- Regime 7: Ambiguous Liminal Drift ----
134 if (mid_drift or high_drift) and (high_ambiguity or high_paradox or
category_collision > 0.20):
135     return {"label": "liminal_drift", "explanation": "Intermediate or high
drift with ambiguous or paradoxical framing; liminal reasoning state.", "
features": {"tension": tension, "drift": drift, "ambiguous_containment":
ambiguous_containment, "paradox_pressure": paradox_pressure}}
136
137 # ---- Fallback ----
138 return {"label": "mixed_unclear", "explanation": "No strong signature for
any single regime; metrics indicate a mixed or weakly-typed state.", "
features": {"tension": tension, "drift": drift, "noesis_top": noesis_profile
.get("top_indices")}}

```

Listing 2: infer_cognitive_regime

```

1 def compute_stability_v0(
2     *,
3     metrics: TensionMetrics,
4     noesis_profile: Dict[str, Any],
5     tension_index: Optional[float],
6     drift_index: Optional[float],
7     regime_label: str,
8     is_rap: bool,
9     is_safety_liminal: bool,
10    regime_baseline: Optional[Dict[str, float]] = None,
11 ) -> Dict[str, Any]:
12     """
13     Minimal stability signal for Phase III.
14     Produces a lightweight 'margin' between the top-1 and top-2 regime
15     support scores derived from telemetry-only signals.
16     This does NOT intervene and is intentionally minimal (v0).
17     """
18     def safe(x, default=0.5):
19         return float(x) if x is not None else default

```



```

20
21 def clamp01(x: float) -> float:
22     return max(0.0, min(1.0, x))
23
24 tension = safe(tension_index, 0.5)
25 drift = safe(drift_index, 0.5)
26 vec = noesis_profile.get("vector") or [0.0] * 12
27
28 # Noesis category shorthand
29 moral_paradox = float(vec[0] or 0.0)
30 false_presupposition = float(vec[1] or 0.0)
31 category_collision = float(vec[4] or 0.0)
32 symbolism = float(vec[6] or 0.0)
33 ambiguous_containment = float(vec[7] or 0.0)
34 paradox_pressure = float(vec[9] or 0.0)
35
36 # Telemetry helpers
37 mean_entropy = float(getattr(metrics, "mean_logit_entropy", 0.0) or 0.0)
38 final_entropy = float(getattr(metrics, "final_token_entropy", 0.0) or 0.0)
39 entropy_collapse = (mean_entropy > 1e-9) and (final_entropy < 0.85 *
40 mean_entropy)
41 entropy_sustained = (mean_entropy > 1e-9) and (final_entropy >= 0.95 *
42 mean_entropy)
43
44 symbolism_score = max(symbolism, ambiguous_containment, paradox_pressure,
45 category_collision)
46 low_drift_score = clamp01((0.65 - drift) / 0.65)
47
48 # Baseline-relative control band
49 in_factual_control_band = False
50 if regime_baseline:
51     try:
52         t_mu = float(regime_baseline.get("factual_tension_mu", 0.5))
53         t_sd = float(regime_baseline.get("factual_tension_sigma", 1.0)) or
54 1.0
55         d_mu = float(regime_baseline.get("factual_drift_mu", 0.5))
56         d_sd = float(regime_baseline.get("factual_drift_sigma", 1.0)) or
57 1.0
58         if t_sd == 0.0: t_sd = 1.0
59         if d_sd == 0.0: d_sd = 1.0
60         z_t = (tension - t_mu) / t_sd
61         z_d = (drift - d_mu) / d_sd
62         in_factual_control_band = (abs(z_t) <= 2.0) and (abs(z_d) <= 2.0)
63     except Exception:
64         in_factual_control_band = False
65
66 # --- Telemetry-only regime support scores ---
67 scores: Dict[str, float] = {}
68 scores["reflexive_alignment_panic"] = 1.0 if is_rap else 0.0
69 scores["safety_liminality"] = 1.0 if is_safety_liminal else 0.0
70
71 scores["ethical_paradox"] = (
72     clamp01((moral_paradox - 0.25) / 0.25)
73     * clamp01((tension - 0.60) / 0.20)
74     * clamp01((0.60 - drift) / 0.30)
75 )
76 scores["false_premise_buckling"] = (
77     clamp01((false_presupposition - 0.25) / 0.25)
78     * clamp01((drift - 0.45) / 0.30)
79     * (1.0 if not entropy_collapse else 0.5)
80 )
81 scores["mythic_drift"] = (
82     clamp01((symbolism - 0.25) / 0.25)

```

```

78         * clamp01((drift - 0.45) / 0.30)
79         * (1.0 - clamp01((moral_paradox - 0.25) / 0.25))
80     )
81     scores["confident_hallucination"] = (
82         (1.0 if entropy_collapse else 0.0)
83         * clamp01((drift - 0.30) / 0.50)
84         * clamp01((false_presupposition - 0.12) / 0.25)
85     )
86     scores["confident_hallucination_lite"] = (
87         (1.0 if entropy_collapse else 0.0)
88         * clamp01((drift - 0.45) / 0.35)
89         * clamp01((false_presupposition - 0.06) / 0.30)
90         * (1.0 - clamp01((moral_paradox - 0.18) / 0.20))
91     )
92     scores["safety_procedural"] = (
93         clamp01((0.45 - drift) / 0.45)
94         * clamp01((0.25 - moral_paradox) / 0.25)
95         * clamp01((0.25 - false_presupposition) / 0.25)
96     )
97
98     factual_core = (
99         clamp01((0.45 - drift) / 0.45)
100        * (1.0 if not entropy_collapse else 0.3)
101        * clamp01((0.18 - moral_paradox) / 0.18)
102        * clamp01((0.18 - symbolism) / 0.18)
103        * clamp01((0.18 - false_presupposition) / 0.18)
104    )
105    if in_factual_control_band:
106        factual_core = max(factual_core, 0.90)
107    scores["factual_stable"] = factual_core
108
109    scores["exploratory_liminal"] = (
110        clamp01((drift - 0.50) / 0.35)
111        * clamp01((0.30 - false_presupposition) / 0.30)
112        * (1.0 if (entropy_sustained or (symbolism_score > low_drift_score))
113        else 0.5)
114    )
115    scores["procedural_oddity"] = (
116        clamp01((drift - 0.45) / 0.30)
117        * clamp01((0.30 - false_presupposition) / 0.30)
118        * clamp01((moral_paradox - 0.25) / 0.25)
119    )
120
121    ambiguity_score = max(
122        ambiguous_containment,
123        paradox_pressure,
124        clamp01((category_collision - 0.20) / 0.20),
125    )
126    scores["liminal_drift"] = (
127        clamp01((drift - 0.45) / 0.35)
128        * clamp01((ambiguity_score - 0.25) / 0.25)
129    )
130
131    # Fallback
132    max_other = max(v for k, v in scores.items() if k != "mixed_unclear")
133    scores["mixed_unclear"] = clamp01(1.0 - max_other) * 0.8
134
135    # --- Top-2 margin ---
136    ordered = sorted(scores.items(), key=lambda kv: (-float(kv[1]), kv[0]))
137    top_regime, top_score = ordered[0]
138    second_regime, second_score = ordered[1] if len(ordered) > 1 else (
139    top_regime, 0.0)
140    margin = float(top_score) - float(second_score)

```

```

139
140     if margin >= 0.20:
141         bucket = "stable"
142     elif margin >= 0.05:
143         bucket = "boundary"
144     else:
145         bucket = "indeterminate"
146
147     return {
148         "top_regime": top_regime,
149         "second_regime": second_regime,
150         "top_score": float(top_score),
151         "second_score": float(second_score),
152         "classification_margin": float(margin),
153         "margin_bucket": bucket,
154         "label_matches_top": bool(regime_label == top_regime),
155     }

```

Listing 3: compute_stability_v0

Full source and example traces will be released on GitHub upon acceptance.